

CASE

Discord Music Bot - Technical Document

A personal Discord music bot written in Python. This document explains the technology used, how each piece works, and the workflow a command follows from typing to sound.

Field	Value
Bot name	CASE
Language	Python 3.13
Framework	discord.py 2.7.1
Audio conversion	FFmpeg 9.0.2 (external binary)
Stream resolution	yt-dlp 2026.08.19
Secret handling	python-dotenv 1.2.4
Voice requirements	PyNaCl 1.6.2, davey 0.1.6
Commands	11 slash commands in 2 cogs
Storage	None - queue lives in memory
Database	None

Contents

1. Why this stack
2. Technology explained
3. Project structure
4. Architecture layers
5. End-to-end workflow
6. The queue mechanism
7. How Discord voice + audio actually work
8. Error handling
9. Known limitations

1. Why this stack

Every choice here exists to keep the project small enough to fully understand. There is no Lavalink server, no Redis, no database, and no cloud service, because none of them are needed yet.

- **discord.py** - the only mature Python library for Discord. It handles the WebSocket connection, the command tree, and the voice protocol.
- **FFmpeg** - Discord accepts only Opus audio. FFmpeg decodes MP3 and re-encodes it, which no Python library does well on its own.

- **yt-dlp** - resolves a search term or URL into a direct audio stream. It is kept strictly behind one function so playback never depends on it.
- **python-dotenv** - loads the bot token from .env so it never sits in code.
- **PyNaCl / davey** - required for voice encryption. discord.py lists these as optional extras, so pip will not install them unless asked.

2. Technology explained

2.1 discord.py

Connects to Discord over a persistent WebSocket. Commands arrive as Interaction objects. The library owns the Gateway session, so CASE never parses Discord's protocol by hand.

2.2 Slash commands and the command tree

Commands are declared with decorators on cog methods. At startup, `tree.sync(guild)` uploads their names, descriptions and parameters to Discord. Global sync can take up to an hour to appear; guild sync is immediate, which suits a single private server.

```
@discord.app_commands.command(name="play", description="Play a song by name, link, or from music/")
@discord.app_commands.describe(song="Song name, a YouTube/SoundCloud link, or a file in music/")
async def play(self, interaction: discord.Interaction, song: str):
    ...
```

2.3 Intents

Intents tell Discord which events to deliver. CASE only uses slash commands, so the default intents are enough and no privileged toggle is required in the Developer Portal. The harmless warning about message content intent can be ignored.

2.4 FFmpeg - the part people miss

FFmpeg is not a Python package. It is a separate program (`ffmpeg.exe` on Windows) that pip cannot install. discord.py launches it as a child process, writes the audio command to its standard input, and reads encoded Opus frames back from standard output. CASE locates the binary itself because a terminal opened before FFmpeg was installed keeps the old PATH.

```
import shutil
from pathlib import Path

FFMPEG_FALLBACK_PATHS = (
    Path(os.environ["LOCALAPPDATA"]) / "Microsoft" / "WinGet" / "Links",
    Path("C:/ffmpeg/bin"),
)

def find_ffmpeg() -> str | None:
    found = shutil.which("ffmpeg")
    if found:
        return found
    for folder in FFMPEG_FALLBACK_PATHS:
        candidate = Path(folder, "ffmpeg.exe")
        if candidate.exists():
            return str(candidate)
    return None
```

2.5 yt-dlp

Given a search phrase or a URL, yt-dlp returns metadata plus a direct, signed audio URL that FFmpeg can stream. It is a blocking library, so CASE runs it in a worker thread to keep the event loop free.

```
track = await asyncio.to_thread(resolve_track, song, user.display_name)
```

2.6 python-dotenv

Reads key/value pairs from `.env` into environment variables. The file is listed in `.gitignore` and is never printed or logged.

3. Project structure

```
CASE/
|-- bot.py           entry point: login, load cogs, sync commands, set presence
|-- config.py        reads .env, exposes MUSIC_DIR and TOKEN
|-- requirements.txt
|-- .env             bot token (git-ignored)
|-- .gitignore
|-- start_case.bat   double-click launcher
|-- music/           local audio files (git-ignored)
`-- cogs/
    |-- general.py    /ping, /help
    `-- music.py      voice, playback, queue, track resolution
```

Only two cog files. A cog is just discord.py's grouping mechanism - splitting non-music commands away from music commands was the only separation that earned its keep. Line counts: bot.py 59, config.py 17, general.py 62, music.py 530.

4. Architecture layers

Responsibilities stay separated so each piece can be replaced alone:

Discord commands	cogs/music.py	Music.play(), Music.pause()
v		
Music player	MusicPlayer	queue, current track, playback loop
v		
Queue	list[Track]	in memory, per guild
v		
Audio source	Track.source	local file path OR remote URL
v		
FFmpeg	FFmpegOpusAudio	mp3/webm -> Opus frames
v		
Discord voice	VoiceClient	encrypted packets to the channel

The important seam

Track is the boundary. It carries a source string that is either a local path or a stream URL. The player only ever reads `.source`, so adding a new source later means writing one resolver function and touching nothing else.

```
class Track:
    def __init__(self, source, title, requested_by="someone",
                  duration=None, webpage_url=None):
        self.source = source          # path OR url
        self.title = title
        ...
```

```
@property
def is_stream(self) -> bool:
    return self.webpage_url is not None
```

Per-guild state

The cog keeps `self.players: dict[int, MusicPlayer]`, keyed by guild id. Two servers therefore get independent queues and independent voice connections.

5. End-to-end workflow

What happens after a user types `/play believer`

1. Discord delivers the interaction

The Gateway WebSocket pushes an Interaction carrying guild id, user and the argument string.

2. Acknowledge immediately

`play()` calls `defer()` first. Discord requires a reply within 3 seconds, and a network lookup can take longer. `defer()` shows "thinking..." and buys time.

3. Resolve to a Track

`resolve_track()` runs in a thread. It first looks in `music/` for a filename match; local files always win. If nothing matches, `yt-dlp` handles the query as a search or a direct URL.

4. Connect to voice

`ensure_player()` checks that the user is in a voice channel, verifies FFmpeg is present, then connects once per guild or moves the existing client.

5. Hand to the player

`player.add(track)` appends to the queue and starts the background loop. The bot replies with a Now Playing embed, or with a queue position if something was already playing.

6. The loop plays it

`_run()` pops the track, builds a `FFmpegOpusAudio` from the source, and hands it to the voice client.

7. Audio flows

FFmpeg decodes the MP3 or pulls the stream, re-encodes to Opus, and writes 20 ms frames. `discord.py` encrypts each frame and sends it to Discord, which plays it for everyone in the channel.

Command lifecycle at a glance

```
user types /play believer
|
v
Interaction arrives -----> play()
|                               |
|                               +--> defer()           "CASE is thinking..."
|                               +--> resolve_track()    to_thread, off the loop
|                               |
|                               | +--> music/believer.mp3 ? use local file
|                               | +--> else yt-dlp search / URL parse
|                               |
|                               +--> ensure_player()
|                               |
|                               +--> user in a voice channel?
```

```

|                                     |      +--> ffmpeg located?
|                                     |      +--> connect or move
|                                     |
|                                     +--> player.add(track)  --> _run() task
|
|--> followup.send()      "Now Playing" embed
|
v
FFmpeg -> Opus -> encrypted packets -> voice channel

```

6. The queue mechanism

One background task drains the queue. It pops a track, plays it, then polls until the track ends - which also covers a pause, because a paused player reports paused rather than finished.

```

while self.queue:
    track = self.queue.pop(0)
    self.current = track
    source = discord.FFmpegOpusAudio(track.source, executable=find_ffmpeg())
    self.voice.play(source)

    while self.voice.is_playing() or self.voice.is_paused():
        await asyncio.sleep(0.5)

    self.current = None

```

Design consequences

- **Auto-advance is free.** When the last frame is sent, `is_playing()` turns false, the inner loop exits, and the next iteration picks up the next track.
- **Skip reuses the same loop.** Calling `voice.stop()` ends playback, which is indistinguishable from a track finishing - so `skip()` only stops the voice and lets the loop continue. If the song was paused, `skip()` resumes first.
- **Stop cancels instead.** `stop()` clears the queue and cancels the task, whose finally block disconnects the voice client.
- **Auto-leave.** When the queue empties normally, the finally block disconnects, and `on_voice_state_update` removes the stale player so the next `/play` starts clean.
- **is_idle() decides the reply.** `/play` checks whether anything is active to choose between a Now Playing embed and a queue position.

7. How Discord voice + audio actually work

The audio pipeline, end to end

```

believer.mp3 on disk          (or a YouTube stream URL)
|
|  FFmpeg process, spawned by discord.py
|  args: -i <source> -f opus -ac 2 -ar 48000 -b:a 128k
v
raw Opus frames, 20 ms each    (~3840 bytes, ~384 bytes after overhead)
|
|  header stripped, frame encrypted (AES-GCM via PyNaCl / libsodium)
|  960 samples per frame @ 48 kHz stereo = Discord's requirement
v
WebSocket -> Discord voice server
|
v

```

decoded, mixed, and played in the voice channel

Why PyNaCl is mandatory

Discord does not accept plain audio. Every Opus frame is encrypted with a session key obtained during the voice handshake. PyNaCl provides the AES-GCM primitive; davey handles the UDP transport. Without them discord.py logs "voice will NOT be supported" and /join fails with a confusing error - so they are listed explicitly in requirements.txt.

Encoding numbers Discord expects

Property	Required value
Sample rate	48000 Hz
Channels	2 (stereo)
Frame size	20 ms
Samples per frame	960
Codec	Opus
Max bitrate	~128 kbps (60 KB/s ceiling)

Reconnection

Streamed audio can drop mid-song. FFmpeg is started with `-reconnect 1 -reconnect_streamed 1 -reconnect_delay_max 5` so it retries a broken connection instead of silently ending the track.

8. Error handling

User-facing messages stay friendly while the full traceback goes to the terminal log, where it is useful for debugging instead of confusing for users.

Situation	CASE replies
User not in a voice channel	You're not in a voice channel.
FFmpeg missing	FFmpeg isn't installed, so I can't play audio. ...
Song / search finds nothing	I couldn't find anything for <code>name</code> .
Nothing is playing	Nothing is playing right now.
Already paused	The music is already paused.
Not paused	The music isn't paused.
Bot not connected	CASE isn't connected to a voice channel.
Skip with empty queue	Nothing left in the queue. Stopping.

One-response rule

A Discord interaction accepts exactly one reply. CASE calls `defer()` first and then replies with `followup()`. `ensure_player()` therefore returns `(player, error)` instead of replying itself - otherwise the interaction would be answered twice and Discord returns "The application did not respond".

```
player, error = await self.ensure_player(interaction)
```

```
if error:
    await interaction.followup.send(error)
    return
```

9. Known limitations

- The queue is in memory. Restarting CASE loses it.
- CASE only runs while the host machine is awake. A VPS is needed for 24/7 use.
- Spotify is unsupported: the official API exposes metadata, not playable audio.
- The SoundCloud extractor in yt-dlp currently returns HTTP 404. Other sources work; this is an upstream yt-dlp issue, and the failure is caught and reported cleanly.
- One MusicPlayer per guild, stored in memory, so state is lost on restart.
- No volume, shuffle, loop, or per-track removal yet - planned for v2.

Planned next

- /remove, /clear, /shuffle, /loop (off / track / queue), /volume
- Button controls on the Now Playing embed
- A pluggable source interface so new resolvers slot in without touching playback
- SQLite for per-server settings such as a DJ role or default channel
- Hosting on a VPS with logging and restart-on-failure